Cấu trúc Dự án NER Tiếng Việt

https://gemini.google.com/app/fd9de83b9d1696b1

User prompt: trình bày cho tôi đầy đủ, chi tiết cấu trúc dự án tốt nhất cho dự án này kèm theo giải thích chi tiết về nhiệm vụ của từng thư mục trong cấu trúc này

Response: Cấu trúc dự án dưới đây được thiết kế theo kiến trúc phân tách mô-đun, cho phép 3 thành viên làm việc song song trên 3 nhánh kỹ thuật (BiLSTM-CRF, Pretrained PLM và LLM Few-shot) cùng báo cáo LaTeX ACL mà không xảy ra xung đột mã nguồn (merge conflict) trên GitHub.

1. Sơ đồ cây thư mục tổng thể

```
vietnamese-ner-acl/
├── .gitignore                      # Loại trừ dữ liệu nặng, trọng số mô hình và file tạm LaTeX
├── README.md                       # Hướng dẫn cài đặt, tải dữ liệu, tái tạo kết quả và link W&B
├── requirements.txt                # Danh sách thư viện Python cố định phiên bản
├── Makefile                        # Các lệnh tắt tự động hóa quy trình chạy thực nghiệm
│
├── configs/                        # Quản lý siêu tham số tập trung (YAML)
│   ├── bilstm_crf.yaml             # Cấu hình nhánh 1 (Thành viên 1)
│   ├── pretrained_plm.yaml         # Cấu hình nhánh 2: PhoBERT & XLM-R (Thành viên 2)
│   └── llm_fewshot.yaml            # Cấu hình nhánh 3: Prompt & In-context learning (Thành viên 3)
│
├── data/                           # Quản lý dữ liệu và kịch bản tải (Không đẩy file nặng lên Git)
│   ├── download_data.sh            # Script tải tự động PhoNER_COVID19, VLSP và trọng số từ Cloud
│   ├── raw/                        # Dữ liệu gốc chưa qua xử lý (.gitignore)
│   ├── processed/                  # Dữ liệu chuẩn hóa JSONL theo định dạng chung (.gitignore)
│   │   ├── phoner_covid19/         # Gồm bản word-level và syllable-level
│   │   └── vlsp/                   # Gồm bản word-level và syllable-level
│   └── splits/                     # Danh sách ID câu kiểm thử cố định để đối chiếu công bằng
│       └── test_indices.json
│
├── src/                            # Mã nguồn chính của dự án
│   ├── __init__.py
│   ├── common/                     # Giao thức dùng chung cho cả 3 thành viên (Phụ trách: TV3)
│   │   ├── schema.py               # Định nghĩa cấu trúc dữ liệu vào/ra và chuẩn nhãn BIO
│   │   ├── metrics.py              # Hàm chấm điểm Entity-level F1 cố định toàn dự án
│   │   ├── logger.py               # Tích hợp Weights & Biases / TensorBoard
│   │   └── word_segmenter.py       # Bộ tách từ tiếng Việt thống nhất (VnCoreNLP / PyVi)
│   │
│   ├── data_prep/                  # Tiền xử lý dữ liệu (Phụ trách: TV1 & TV2)
│   │   ├── process_phoner.py       # Chuẩn hóa PhoNER_COVID19 (Thành viên 1)
│   │   └── process_vlsp.py         # Chuẩn hóa VLSP 2016/2018 (Thành viên 2)
│   │
│   ├── models/                     # Ba nhánh mô hình độc lập (Tránh xung đột Git)
│   │   ├── bilstm_crf/             # Nhánh 1: Tự cài đặt từ đầu (Phụ trách: TV1)
│   │   │   ├── vocab.py            # Bộ từ điển Word & Character
│   │   │   ├── dataset.py          # DataLoader theo lô (batch padding & mask)
│   │   │   ├── char_encoder.py     # Tầng biểu diễn ký tự (Char-CNN / Char-LSTM)
│   │   │   ├── crf.py              # Tầng CRF tự cài (Forward-Backward & Viterbi decoding)
│   │   │   ├── model.py            # Kiến trúc tích hợp BiLSTM-CRF
│   │   │   ├── train.py            # Vòng lặp huấn luyện và thực nghiệm cắt bỏ (ablation)
│   │   │   └── predictor.py        # Hàm suy luận chuẩn hóa phục vụ Demo
│   │   │
│   │   ├── pretrained/             # Nhánh 2: Tinh chỉnh PhoBERT & XLM-R (Phụ trách: TV2)
│   │   │   ├── subword_align.py    # Căn chỉnh nhãn từ gốc - từ phụ và gán nhãn -100
│   │   │   ├── model.py            # Kiến trúc ghép nối các tầng ẩn cuối Transformer
│   │   │   ├── trainer.py          # Chiến lược đóng băng (freezing), scheduler, weight decay
│   │   │   ├── attention_viz.py    # Trích xuất và vẽ bản đồ trọng số chú ý (Attention Map)
│   │   │   └── predictor.py        # Hàm suy luận chuẩn hóa phục vụ Demo
│   │   │
│   │   └── llm_ner/                # Nhánh 3: Few-shot In-Context Learning (Phụ trách: TV3)
│   │       ├── prompt_templates.py # Khung câu lệnh Zero-shot và Few-shot
│   │       ├── retriever.py        # Thuật toán chọn mẫu tương đồng (k-NN / BM25) và ngẫu nhiên
│   │       ├── api_client.py       # Gọi LLM và đo thời gian suy luận
│   │       ├── postprocess.py      # Hậu xử lý chuỗi văn bản sinh ra về chuẩn nhãn BIO
│   │       └── predictor.py        # Hàm suy luận chuẩn hóa phục vụ Demo
```

```
│   │
│   └── analysis/                    # Phân tích lỗi chéo và trực quan hóa (Phụ trách: TV3)
│       ├── error_taxonomy.py        # Phân loại lỗi biên thực thể (boundary) và nhập nhằng nhãn
│       ├── confusion_matrix.py      # Vẽ ma trận nhầm lẫn đối chiếu 3 nhánh
│       └── compare_granularity.py   # Biểu đồ so sánh mức từ (word) và mức âm tiết (syllable)
│
├── outputs/                         # Lưu kết quả xuất ra để ghép báo cáo
│   ├── predictions/                 # Tệp dự đoán trên tập test theo định dạng thống nhất
│   │   ├── bilstm_crf_test.jsonl    # Do Thành viên 1 xuất cuối Tuần 4
│   │   ├── pretrained_test.jsonl    # Do Thành viên 2 xuất cuối Tuần 4
│   │   └── llm_test.jsonl           # Do Thành viên 3 xuất cuối Tuần 4
│   ├── figures/                     # Hình ảnh biểu đồ F1, Attention Map xuất thẳng sang LaTeX
│   ├── tables/                      # Bảng kết quả dạng mã LaTeX tự động sinh
│   └── checkpoints/                 # Trọng số mô hình đã huấn luyện (.gitignore)
│
├── scripts/                         # Kịch bản tái tạo kết quả tự động (Yêu cầu bắt buộc)
│   ├── reproduce_main_table.sh      # Chạy và xuất bảng kết quả chính trong báo cáo
│   └── run_ablations.sh             # Chạy toàn bộ thí nghiệm cắt bỏ của Tuần 4
│
├── demo/                            # Ứng dụng trình diễn trực quan ở Tuần 6 (Phụ trách: TV3)
│   └── app.py                       # Giao diện Gradio/Streamlit so sánh trực tiếp 3 mô hình
│
└── report/                          # Báo cáo chuẩn ACL và Slide thuyết trình (XeLaTeX/LuaLaTeX)
    ├── acl_latex/                   # Mã nguồn LaTeX chia theo từng tệp độc lập
    │   ├── acl.sty                  # Gói định dạng chính thức của hội nghị ACL
    │   ├── main.tex                 # Tệp gốc khai báo \usepackage[final]{acl} và ghép các mục
    │   ├── references.bib           # Quản lý trích dẫn BibTeX chung
    │   ├── sections/                # Tách nhỏ từng mục để 3 người viết cùng lúc không đụng độ
    │   │   ├── 00_abstract.tex      # Thành viên 3
    │   │   ├── 01_intro.tex         # Thành viên 1 (Bài toán) & Thành viên 3
    │   │   ├── 02_related_work.tex  # Thành viên 3
    │   │   ├── 03_method.tex        # TV1 (BiLSTM-CRF), TV2 (PLM), TV3 (LLM)
    │   │   ├── 04_experiments.tex   # Thành viên 2 (Dữ liệu, Thiết lập, Bảng chính)
    │   │   ├── 05_analysis.tex      # TV1 (Ablation), TV2 (Attention), TV3 (Phân tích lỗi)
    │   │   ├── 06_conclusion.tex    # Thành viên 3
    │   │   ├── 07_limitations.tex   # Bắt buộc, nằm ngoài giới hạn 8 trang (Thành viên 3)
    │   │   ├── 08_contributions.tex# Bắt buộc, ghi rõ tỉ lệ 33.3% - 33.3% - 33.4% và khai báo AI
    │   │   └── 09_appendix.tex      # Phụ lục tùy chọn
    │   └── milestones/              # Lưu trữ bản nộp Proposal (2 trang) và Milestone (4 trang)
    └── slides/                      # Slide thuyết trình LaTeX Beamer (10-12 phút)
```

2. Giải thích chi tiết nhiệm vụ của từng thư mục

Nhóm thư mục gốc và cấu hình (`configs/`, `scripts/`)

- **`.gitignore` và `data/download_data.sh`:** Quy định môn học cấm đưa dữ liệu lớn hoặc trọng số mô hình (`checkpoints/`) trực tiếp lên kho GitHub. Tệp `.gitignore` sẽ chặn các thư mục `data/raw/`, `data/processed/` và `outputs/checkpoints/`, trong khi `download_data.sh` chứa lệnh tự động tải dữ liệu và trọng số từ Google Drive/Hugging Face về đúng vị trí khi giảng viên chấm bài.
- **`configs/`:** Tách biệt hoàn toàn siêu tham số (learning rate, dropout, hidden dimension, số mẫu few-shot) ra khỏi mã nguồn Python. Khi thực hiện tinh chỉnh siêu tham số và thí nghiệm cắt bỏ ở Tuần 4, các thành viên chỉ cần tạo/sửa file YAML và đẩy log lên Weights & Biases mà không làm xáo trộn lịch sử code.
- **`scripts/`:** Đáp ứng yêu cầu bắt buộc về khả năng tái tạo kết quả (reproducibility). Giảng viên chỉ cần chạy một lệnh `bash scripts/reproduce_main_table.sh` là hệ thống tự động nạp dữ liệu kiểm thử, chạy suy luận trên 3 nhánh và in ra bảng điểm F1 khớp với báo cáo.

Nhóm thư mục dữ liệu (`data/`)

- **`data/processed/`:** Lưu trữ hai tập dữ liệu `PhoNER_COVID19` (do Thành viên 1 xử lý) và `VLSP` (do Thành viên 2 xử lý) đã được đưa về cùng một cấu trúc tệp JSONL thống nhất từ đầu Tuần 1. Mỗi tập dữ liệu đều có hai phiên bản: mức từ (`word-level`) và mức âm tiết (`syllable-level`) để phục vụ phần mở rộng ở Tuần 5.
- **`data/splits/test_indices.json`:** Giữ danh sách chỉ số (ID) các câu kiểm thử cố định. Nhờ tệp này, khi Thành viên 1 và Thành viên 2 so sánh mức từ và mức âm tiết ở Tuần 4–5, cả hai nhánh đảm bảo đánh giá trên cùng một tập thực thể đích.

Nhóm thư mục mã nguồn (`src/`)

- **`src/common/` (Hạ tầng lõi – Thành viên 3):** Là "hợp đồng giao tiếp" giữa 3 thành viên.
    - `schema.py` định nghĩa định dạng câu đầu vào và chuỗi nhãn BIO chuẩn để Thành viên 3 có thể tạo dữ liệu giả lập (dummy data) viết khung đánh giá ngay trong Tuần 1 mà không cần đợi Thành viên 1 và 2 xử lý xong dữ liệu thật.
    - `metrics.py` đóng băng cách tính điểm Entity-level F1 cho từng loại thực thể ngay cuối Tuần 1; cả 3 nhánh bắt buộc gọi hàm này để đảm bảo số liệu trên toàn báo cáo hoàn toàn đồng nhất.
    - `word_segmenter.py` cố định một công cụ tách từ duy nhất để tránh lệch biên từ khi đối chiếu giữa các nhánh ở Tuần 4.
- **`src/models/bilstm_crf/` (Nhánh 1 – Thành viên 1):** Chứa toàn bộ phần tự cài đặt từ đầu để ăn trọn 25% điểm kỹ thuật và phục vụ kiểm tra đạo văn MOSS/JPlag cũng như vấn đáp cá nhân. Việc tách riêng `char_encoder.py` (biểu diễn ký tự bằng CNN/LSTM) và `crf.py` (thuật toán quy hoạch động Forward-Backward tính hàm loss và Viterbi tìm chuỗi nhãn tối ưu) giúp Thành viên 1 dễ dàng bật/tắt từng thành phần khi chạy thí nghiệm cắt bỏ (Ablation study) ở Tuần 4 và trả lời câu hỏi vấn đáp về từng hàm.
- **`src/models/pretrained/` (Nhánh 2 – Thành viên 2):** Tập trung giải quyết bài toán kỹ thuật đặc thù của mô hình Transformer trên tiếng Việt. Tệp `subword_align.py` xử lý việc một từ gốc bị tách thành nhiều từ phụ (subwords) và gán nhãn bỏ qua (`-100`) tại hàm Cross-Entropy Loss cho các từ phụ đứng sau. Tệp `attention_viz.py` trích xuất ma trận trọng số chú ý từ tầng cuối để xuất hình ảnh trực quan hóa cho mục Analysis.
- **`src/models/llm_ner/` (Nhánh 3 – Thành viên 3):** Thực hiện phần mở rộng với mô hình ngôn ngữ lớn. Tách bạch rõ khâu chọn mẫu đưa vào ngữ cảnh (`retriever.py` so sánh chọn ngẫu nhiên và chọn theo độ tương đồng câu) và khâu hậu xử lý (`postprocess.py` ép văn bản tự do do LLM sinh ra về đúng chuỗi nhãn BIO hợp lệ và thống kê tỷ lệ sinh sai định dạng).
- **`predictor.py` (Nằm ở cuối mỗi nhánh mô hình):** Đầu Tuần 6, mỗi thành viên bọc mô hình của mình vào một hàm chuẩn `predict(sentence: str) -> List[Dict]` trong tệp này. Thành viên 3 chỉ cần `import` 3 hàm này vào `demo/app.py` để dựng giao diện trình diễn mà không cần can thiệp vào mã huấn luyện bên trong của bạn cùng nhóm.

Nhóm thư mục xuất kết quả và phân tích (`outputs/`, `src/analysis/`)

- **`outputs/predictions/`:** Cuối Tuần 4, cả 3 thành viên xuất kết quả dự đoán trên tập kiểm thử ra 3 tệp JSONL có định dạng giống hệt nhau tại đây. Thiết kế này cho phép Thành viên 3 viết sẵn kịch bản phân tích lỗi (`src/analysis/`) trên tệp dự đoán của nhánh LLM trước, sau đó chỉ việc nạp thêm 2 tệp của Thành viên 1 và 2 ngay khi nhận bàn giao ở Tuần 5 để vẽ ma trận nhầm lẫn và phân tích lỗi biên thực thể.

Nhóm thư mục báo cáo ACL (`report/acl_latex/`)

- **Tách nhỏ thư mục `sections/`:** Nếu cả 3 người cùng sửa một tệp `main.tex` vào ngày sát hạn nộp Proposal (Tuần 1), Milestone (Tuần 3) hay Báo cáo cuối (Tuần 6), xung đột Git sẽ rất dễ xảy ra. Việc dùng lệnh `\input{sections/...}` trong `main.tex` và chia thành các tệp `.tex` riêng biệt theo đúng phân công giúp Thành viên 1 viết mục Phương pháp truyền thống (`03_method.tex`), Thành viên 2 viết mục Thực nghiệm (`04_experiments.tex`), và Thành viên 3 viết mục Phân tích & Giới hạn (`05_analysis.tex`, `07_limitations.tex`) hoàn toàn độc lập.
- **Kiểm soát quy chuẩn ACL:** Tệp `07_limitations.tex` và `08_contributions.tex` (ghi rõ phân công, tỷ lệ đóng góp và khai báo sử dụng công cụ AI) được tách riêng đặt sau mục Kết luận để Thành viên 3 dễ dàng kiểm tra ràng buộc tối đa 8 trang nội dung chính trước khi biên dịch ra tệp `NhomXX_DeTai07.pdf` bằng XeLaTeX/LuaLaTeX.
